SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

PRACTICAL RECORD NOTE

STUDENT NAME :

REGISTER NUMBER :

CLASS & SECTION : MCA GEN AI – C

YEAR & SEMESTER : I Year & II Semester

SUBJECT CODE : PCA25C07J

SUBJECT TITLE : ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

APRIL 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM Nagar, Kattankulathur – 603 203

CERTIFICATE

Certified to be the bonafide record of practical work done by _____ Register No._____ of MCA GEN AI Degree course for PCA25C07J – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING in the Computer lab in SRM Institute of Science and Technology during the academic year 2025-2026 .

Staff In-Charge *Head of the Department*

Submitted for Semester Practical Examination held on _____.

Internal Examiner *External Examiner*

S.NO.	DATE	TITLE OF THE EXPERIMENT	PAGE NO.	STAFF SIGN.
1.	03.12.25	Water Jug Problem Using Breadth-First Search		
2.	10.12.25	Uninformed Search Using Breadth-First Search(BFS)		
3.	17.12.25	Uninformed Search Using Depth-First Search(DFS)		
4.	05.01.26	Hill Climbing Algorithm		
5.	12.01.26	Model-Based Traffic Light Controller Agent		
6.	21.01.26	Finding Shortest Path Using A* Algorithm		
7.	29.01.26	Medical Diagnosis Expert System Using Prolog		
8.	05.02.26	Student Performance Evaluation Expert System Using Prolog		
9.	12.02.26	Animal Identification Expert System Using Prolog		
10.	19.02.26	Student Score Prediction Using Linear Regression		
11.	26.02.26	Iris Flower Classification Using Logistic Regression		
12.	06.03.26	Customer Purchase Prediction Using Decision Tree		
13.	12.03.26	Iris Flower Classification Using Support Vector Machine (SVM)		
14.	20.03.26	Customer Segmentation Using K-Means Clustering		

EX.NO:1
DATE:

WATER JUG PROBLEM

AIM:

To write a python program to execute the water jug problem in artificial intelligence.

PROCEDURE:

Production rules for solving the water jug problem
Here, let x denote the 4-gallon jug and y denote the 3-gallon jug.

S.No.	Initial State	Condition	Final state	Description of action taken
1.	(x,y)	If x<4	(4,y)	Fill the 4 gallon jug completely.
2.	(x,y)	If y<3	(x,3)	Fill the 3 gallon jug completely.
3.	(x,y)	If x>0	(x-d,y)	Pour some part from the 4 gallon jug.
4.	(x,y)	If y>0	(x,y-d)	Pour some part from the 3 gallon jug.
5.	(x,y)	If x>0	(0,y)	Empty the 4 gallon jug.
6.	(x,y)	If y>0	(x,0)	Empty the 3 gallon jug.
7.	(x,y)	If (x+y)<7	(4, y-[4-x])	Pour some water from the 3 gallon jug fill the four gallon jug.
8.	(x,y)	If (x+y)<7	(x-[3-y],y)	Pour some water from the 4 gallon jug to fill the 3 gallon jug.
9.	(x,y)	If (x+y)<4	(x+y,0)	Pour all water from 3 gallon jug to the 4 gallon jug.
10.	(x,y)	If (x+y)<3	(0, x+y)	Pour all water from the 4 gallon jug to the 3 gallon jug.

Solution of water jug problem according to the production rules:

S.No.	4 gallon jug contents	3 gallon jug contents	Rule followed
1.	0 gallon	0 gallon	Initial state
2.	0 gallon	3 gallons	Rule no.2
3.	3 gallons	0 gallon	Rule no. 9
4.	3 gallons	3 gallons	Rule no. 2
5.	4 gallons	2 gallons	Rule no. 7
6.	0 gallon	2 gallons	Rule no. 5
7.	2 gallons	0 gallon	Rule no. 9

SOURCE CODE:

```
x=int(input("enter the value of X:"))
y=int(input("enter the value   of Y:"))
while True:
    rno=int(input("enter the rule number:"))
    #fill jar 1
    if rno==1:
        if x<4: x=4
    #fill jar 2
     if rno==2:
        if y<3: y=3
    #empty jar 1
    if rno==5:
        if x>0: x=0
    #empty jar 2
    if rno==6:
        if y<0: y=0
    #pour water from jar2 to fill jar1
    if rno==7:
        if x+y>=4 and y>0: x,y=4,y-(4-x)
   if rno==8:
        if x+y>=3 and y>0: x,y=x-(3-y),3
    #pour water from jar2 to jar1
    if rno==9:
        if x+y<=4 and y>0: x,y=x+y,0
    #pour water from jar1 to fill jar2
    if rno==10:
        if x+y<=3 and y>0: x,y=2,(x+y)
    print("the value of x:",x)
    print("the value of y:",y)
    if(x==2):
        print("this is the goal state")
        break
```

OUTPUT:

```
enter the value of X:0
enter the value of Y:0
enter the rule number:2
the value of x: 0
the value of y: 3
enter the rule number:9
the value of x: 3
the value of y: 0
enter the rule number:2
the value of x: 3
the value of y: 3
enter the rule number:7
the value of x: 4
the value of y: 2
enter the rule number:5
the value of x: 0
the value of y: 2
enter the rule number:9
the value of x: 2
the value of y: 0
this is the goal state
```

RESULT:

The given program "Water jug problem" is executed and the output is verified.

EX.NO:2

DATE:

BREADTH FIRST SEARCH

AIM:

To write a python program to execute the Breadth first search problem in artificial

intelligence .

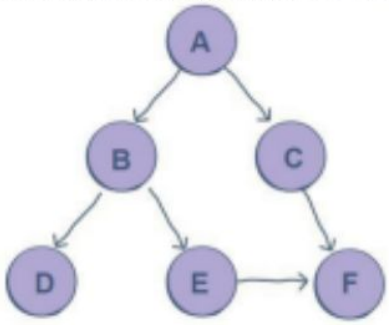
PROCEDURE:

STEP-1:

1. Pick any node, visit the adjacent unvisited vertex, mark it as visited, display it, and insert it in a queue.
2. If there are no remaining adjacent vertices left, remove the first vertex from the queue
3. Repeat step 1 and step 2 until the queue is empty or the desired node is found.

STEP-2:

Consider the graph, which is implemented in the code below:



STEP-3: The illustrated graph is represented using an adjacency list. An easy way to do this in Python is to use a dictionary data structure, where each vertex has a stored list of its adjacent nodes.

STEP-4: visited is a list that is used to keep track of visited nodes.

STEP-5: queue is a list that is used to keep track of nodes currently in the queue.

STEP-6: The arguments of the bfs function are the visited list, the graph in the form of a dictionary, and the starting node A.

STEP-7: bfs follows the algorithm described above:

1. It checks and appends the starting node to the visited list and the queue.
2. Then, while the queue contains elements, it keeps taking out nodes from the queue, appends the neighbors of that node to the queue if they are unvisited, and marks them as visited.
3. This continues until the queue is empty.

SOURCE CODE:

```
graph={
   '5':['3','7'],
   '3':['2','4'],
   '7':['8'],
   '2':[],
   '4':['8'],
   '8':[]
}

visited=[]  # List for visited nodes
queue=[]   #Initialize a queue

def bfs(visited,graph,node):   #Function of BFS
  visited.append(node)
  queue.append(node)

  while queue:    #Creating loop to visit each node
    m=queue.pop(0)
    print(m,end="")

    for neighbour in graph[m]:
      if neighbour not in visited:
        visited.append(neighbour)
        queue.append(node)
        while queue:
          m=queue.pop(0)
          print(m,end="")
          for neighbour in graph[m]:
            if neighbour not in visited:
              visited.append(neighbour)
              queue.append(neighbour)
#Driver code
print("\n following is the breadth-first search")
bfs(visited,graph,'5')      #Function calling
```

OUTPUT:

following is the breadth-first search 5578

RESULT:

The program “Depth First Search” is executed and the output is verified.

EX.NO:3
DATE:

DEPTH FIRST SEARCH

AIM:

To write a python program to execute the Depth first search problem in artificial intelligence .

PROCEDURE:

Step 1: Create an adjacency list for the above graph

Step 2: Create an empty stack
.
Step 3: Push 'A' into the stack.

Step 4: Pop 'A' and push 'B' and 'F'. Mark node 'A' as the visited node.

Step 5: pop 'F' and push 'D'. Mark 'F' as a visited node.

Step 6: pop 'D' and push 'C'. Mark 'D' as a visited node.

Step 7: pop 'C' and push 'E'. Mark 'C' as a visited node.

SOURCE CODE:

```
graph = {
'A' : ['B','C'],
'B' : ['D', 'E'],
'C' : ['F'],
'D' : [],
'E' : ['F'],
'F' : []
}
visited = set() # Set to keep track of visited nodes.
def dfs(visited, graph, node):
	if node not in visited:
		print (node)
		visited.add(node)
		for neighbour in graph[node]:
			dfs(visited, graph, neighbour)
dfs(visited, graph, 'A')
```

OUTPUT:

A
B
D
E
F
C

RESULT:

The given program “Depth First Search” is executed successfully and the output is verified

EX.NO:4
DATE:

Hill Climbing Algorithm

AIM:

The aim of the hill climbing algorithm is to find the peak of a hill or the optimum solution of a problem by iteratively improving a candidate solution.

PROCEDURE:

1. **Initialization:**
   - Start with an initial solution, often chosen randomly or based on some heuristic.

2. **Evaluation:**
   - Evaluate the current solution by calculating its objective function value.

3. **Neighbour Generation:**
   - Generate neighbouring solutions by making small modifications to the current solution. These modifications could involve changing one or more parameters.

4. **Selection:**
   - Select the best neighbouring solution based on the objective function value. If the objective function value of the neighbour is better than the current solution, move to that neighbour; otherwise, stay at the current solution.

5. **Termination:**
   - Repeat steps 2-4 until a termination condition is met. This condition could be reaching a maximum number of iterations, achieving a certain objective function value, or other criteria.

SOURCE CODE:

```
Import numpy as np
def find_neighbours(state,landscape):
        Neighbours=[]
        Dim=landscape.shape
        #left neighbour
        If state[0]!=0:
                Neighbours.append((state[0]-1,state[1]))
        #right neighbour
        If state[0]!= dim[0]-1:
                Neighbours.append((state[0]+1,state[1]))
        #top neighbour
        If state[1]!=0:
                Neighbours.append((state[0],state[1]-1))
        #bottom neighbour
        If state[1]!=dim[1]-1:
                Neighbours.append((state[0],state[1]+1))
        #top left
        If state[0]!=0 and state[1]!=0:
                Neighbours.append((state[0]-1,state[1]-1))
        #bottom left
        If state[0]!=0 and state[1]!= dim[1]-1:
                Neighbours.append((state[0]-1,state[1]+1))
        #top right
        If state[0]!=dim[0]-1 and state[1]!=0:
                Neighbours.append((state[0]+1,state[1]-1))
        #bottom right
        If state[0]!=dim[0]-1 and state[1]!=dim[1]-1:
                Neighbours.append((state[0]+1,state[1]+1))
        Return neighbours
# current optimization objective:loca;/global maximum

def hill_climb(curr_state,landscape):
        Neighbours=find_neighbours(curr_state,landscape)
        Bool
        Ascended=False
        Next_state=curr_state
        For neighbour in neighbours:
                If Landscape[neighbour[0]][neighbour[1]]>landscape[next_state[0]][next_state[1]]:
                Next_state=neighbour
                Ascended=True
        Return ascended,next_state
```

```
def __main__():
        Landscape=np.random.randint(1,high=50,size=(10,10))
        Print(landscape)
        Start_state=(3,6)
        Current_state=start_state
        Count=1
        Ascending=True
        While ascending:
                Print("\nstep #",count)
                Print("current state coordinates: ",current_state)
                Print("current state Value:",landscape[current_state[0]][current_state[1]])
        Count += 1
        Ascending,current_state =Hill_climb(current_state,landscape)
        Print("\n step #",count)
        Print("optimization objective reached")
        Print("final state coordinates: ",current_state)
        Print("final state Value:",landscape[current_state[0]][current_state[1]])
__main__()
```

OUTPUT:

```
[[18 6 40 9 1 34 25 49 4 17]
[ 2 24 45 6 34 8 24 7 27 10]
[ 5 34 30 33 35 43 2 43 12 27]
[ 6 15 9 17 40 6 5 34 47 2]
[ 2 30 39 28 24 27 17 15 46 20]
[ 7 14 26 3 12 19 29 3 5 20]
[11 1 48 19 32 26 19 19 26 1]
[41 21 19 21 44 37 39 7 36 30]
[36 22 46 27 44 34 42 31 45 44]
[21 48 16 21 23 28 41 16 40 4]]

Step # 1
Current state coordinates: (3, 6)
Current state value: 5
Step # 2
Optimization objective reached
Final state coordinates: (2, 5)
Final state value: 43
Step # 3
Optimization objective reached
Final state coordinates: (2, 5)
Final state value: 43
```

RESULT:

The given program “Hill Climbing Algorithm” is executed successfully and the output is verified

Ex No:5	Model-Based Traffic Light Controller Agent

Aim

To design and implement a Model-Based Intelligent Agent for a traffic light system that makes decisions using both current sensor input and past memory state.

Objectives

- To understand model-based agent architecture
- To implement internal memory for decision making
- To simulate agent behavior in partially observable environments
- To compare reflex and model-based decisions

Theory

A Model-Based Agent is an intelligent agent that maintains internal memory of the environment and uses previous states to make decisions when current input is uncertain.

Working Principle:

Perception → Update Internal Model → Decide Action → Execute

Problem Description

A traffic light system generates signals such as Red (R), Yellow (Y), and Green (G), corresponding to STOP, SLOW, and GO actions. Sometimes, the signal may be invalid or unclear. The problem is to design a model-based agent that uses both current input and stored previous state to make correct decisions even in uncertain conditions.

Algorithm

1. Start
2. Initialize memory (previous_state = None)
3. Read current traffic signal
4. If valid (R/Y/G), update memory
5. Else, use stored memory
6. Apply rules: R→STOP, Y→SLOW, G→GO
7. Print output

8. Repeat

9. Stop

Source Code

```
class TrafficAgent:
    def __init__(self):
        self.previous_state = None

    def decide_action(self, light):
        light = light.upper()

        if light in ["R", "Y", "G"]:
            self.previous_state = light
        else:
            light = self.previous_state
        if light == "R":
            return "STOP - Red Light (stored or sensed)"
        elif light == "Y":
            return "SLOW DOWN - Yellow Light (caution)"
        elif light == "G":
            return "GO - Green Light (proceed)"
        else:
            return "NO ACTION - No previous data available"
agent = TrafficAgent()
signals = ["G", "Y", "BLUR", "R", "UNKNOWN", "G"]
for s in signals:
    print("Signal:", s, "→ Action:", agent.decide_action(s))
```

Output

```
Signal: G → Action: GO - Green Light (proceed)
Signal: Y → Action: SLOW DOWN - Yellow Light (caution)
Signal: BLUR → Action: SLOW DOWN - Yellow Light (caution)
Signal: R → Action: STOP - Red Light (stored or sensed)
Signal: UNKNOWN → Action: STOP - Red Light (stored or sensed)
Signal: G → Action: GO - Green Light (proceed)
```

Result

The model-based agent successfully controlled the traffic signal by using both current sensor input and stored memory of previous states. When invalid signals were received, the agent relied on its internal state to make correct decision

Ex No:6	Finding Shortest Path Using A* Algorithm

Aim

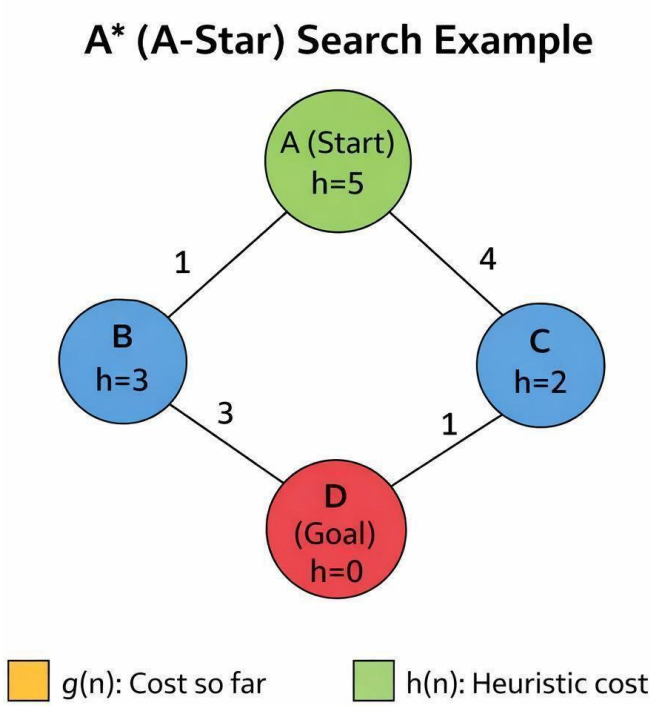
To implement the A* (A-star) search algorithm to find the shortest path between a start node and a goal node in a weighted graph using heuristic values.

Objectives

- To understand informed search strategies.
- To implement A* using the evaluation function f(n) = g(n) + h(n).
- To compute the optimal path from source to destination.

Theory

A* is an informed best-first search algorithm used for pathfinding and graph traversal. It uses two cost functions: g(n), the actual cost from the start node to node n, and h(n), the heuristic estimate from node n to the goal. The total estimated cost is f(n) = g(n) + h(n). The algorithm always expands the node with the lowest f(n) value.



4. Algorithm

1. Initialize open list with the start node.
2. Set g_cost(start) = 0 and create parent dictionary.
3. While open list is not empty:
   - Select node with minimum f(n) = g(n) + h(n)
   - If it is the goal node, reconstruct the path
   - Otherwise explore its neighbors
   - Update g_cost and parent if a better path is found
   - Add neighbor to open list if not already present
4. Return the shortest path

Source Code:

```
def a_star(graph, heuristic, start, goal):
    open_list = [start]
    g_cost = {start: 0}
    parent = {}

    while open_list:
        current = min(open_list, key=lambda node: g_cost[node] + heuristic[node])
        open_list.remove(current)

        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()
            return path

        for neighbor, cost in graph[current]:
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                parent[neighbor] = current
```

```
            if neighbor not in open_list:
                open_list.append(neighbor)

    return None

graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 3)],
    'C': [('D', 1)],
    'D': []
}

heuristic = {
    'A': 5,
    'B': 3,
    'C': 2,
    'D': 0
}

start = 'A'
goal = 'D'

path = a_star(graph, heuristic, start, goal)

print("Shortest Path:", path)
```

Output

```
Shortest Path: ['A', 'B', 'D']
```

Result

The A* algorithm was successfully implemented and the optimal path from A to D was obtained using heuristic-based informed search

Ex No: 7	Medical Diagnosis Expert System Using Prolog

Aim

To design and implement a simple rule-based Medical Diagnosis Expert System using Prolog to identify diseases based on symptoms.

Objectives

- To understand rule-based programming in Prolog.
- To implement facts, rules, and dynamic predicates.
- To demonstrate unification and backtracking concepts.
- To simulate a simple expert system.

Theory

An Expert System is an Artificial Intelligence application that uses knowledge and inference rules to solve problems in a specific domain. Prolog is a logic programming language based on First Order Predicate Logic. It uses facts, rules, and queries to derive conclusions.

This medical expert system diagnoses diseases based on user-provided symptoms. It uses dynamic predicates to store responses and logical rules to infer diseases.

Problem Description

In real-world scenarios, diagnosing diseases requires analyzing multiple symptoms and matching them with medical knowledge. This problem focuses on building a simple expert system that takes user input in the form of symptoms and identifies the possible disease using predefined rules. The system simulates basic medical reasoning using Prolog's logical inference capabilities.

Algorithm

1. Start the program.
2. Ask the user about symptoms.
3. Store user responses dynamically.
4. Match symptoms with predefined disease rules.
5. Display the diagnosed disease.
6. Clear stored responses.

Source Code:

```
:- dynamic symptom/1.
```

```
start :-
    write(' ---------------------------------- '), nl,
    write(' MEDICAL DIAGNOSIS EXPERT SYSTEM '), nl,
    write(' ---------------------------------- '), nl,
    nl,
    diagnose(Disease),
    write('You may have: '), write(Disease), nl,
    nl,
    write('Note: This is only a basic expert system.'), nl,
    write('Please consult a doctor for proper diagnosis.'), nl,
    undo.

diagnose(fever) :-
    symptom(high_temperature),
    symptom(body_pain),
    symptom(headache).

diagnose(common_cold) :-
    symptom(runny_nose),
    symptom(sneezing),
    symptom(sore_throat).

diagnose(malaria) :-
    symptom(high_temperature),
    symptom(chills),
    symptom(sweating).

diagnose(typhoid) :-
    symptom(high_temperature),
    symptom(weakness),
    symptom(stomach_pain).
diagnose(covid19) :-
    symptom(fever),
    symptom(dry_cough),
    symptom(loss_of_taste).
symptom(Symptom) :-
    (   known(yes, Symptom)
```

```
    -> true
    ; known(no, Symptom)
    -> fail
    ; ask(Symptom)
    ).
ask(Symptom) :-
    write('Do you have '), write(Symptom), write('? (yes/no): '),
    read(Response),
    nl,
    asserta(known(Response, Symptom)),
    Response = yes.
undo :-
    retract(known(_, _)),
    fail.
undo.
```

Output

```
-----------------------------------
 MEDICAL DIAGNOSIS EXPERT SYSTEM
-----------------------------------

Do you have high_temperature? (yes/no): yes.

Do you have body_pain? (yes/no): |: yes.

Do you have headache? (yes/no): |: yes.

You may have: fever

Note: This is only a basic expert system.
Please consult a doctor for proper diagnosis.
true .
```

Result

Thus, the Medical Diagnosis Expert System was successfully implemented using Prolog and tested with sample inputs.

Ex No: 8	Student Performance Evaluation Expert System Using Prolog

Aim

To design and implement a rule-based expert system using Prolog to evaluate student performance based on academic inputs.

Objectives

• To understand expert system concepts using Prolog.
• To apply facts and rules for decision-making.
• To evaluate student performance based on predefined criteria.
• To demonstrate logical inference and backtracking.

Theory

An Expert System simulates human decision-making using rules and knowledge. This system evaluates student performance using attendance, marks, and participation.

Problem Description

The problem is to develop an expert system that evaluates a student's overall performance based on inputs such as marks, attendance, and participation. The system should ask relevant questions, process responses using predefined rules, and classify the student into categories like excellent, good, average, or poor.

Algorithm

1. Start the program.
2. Ask the user about student details.
3. Store responses dynamically.
4. Apply rules to evaluate performance.
5. Display performance result.
6. Clear stored data.

Source Code:

```
:- dynamic known/2.

start :-
write(' ---------------------------------- '), nl,
write(' STUDENT PERFORMANCE EXPERT SYSTEM '), nl,
```

```
write(' ---------------------------------- '), nl,
nl,
evaluate(Result),
write('Student Performance: '), write(Result), nl,
undo.

evaluate(excellent) :-
response(high_marks),
response(high_attendance),
response(active_participation).

evaluate(good) :-
response(moderate_marks),
response(high_attendance).

evaluate(average) :-
response(moderate_marks),
response(low_attendance).

evaluate(poor) :-
response(low_marks).

response(Q) :-
(known(yes, Q) -> true ;
known(no, Q) -> fail ;
ask(Q)).

ask(Q) :-
write('Does the student have '), write(Q), write('? (yes/no): '),
read(R), nl,
assert(known(R, Q)),
R = yes.

undo :- retract(known(_, _)), fail.
undo.
```

Output

```
-----------------------------------
 STUDENT PERFORMANCE EXPERT SYSTEM
-----------------------------------

Does the student have high_marks? (yes/no): yes.

Does the student have high_attendance? (yes/no): |: yes.

Does the student have active_participation? (yes/no): |: yes.

Student Performance: excellent
true .

23 ?-
```

Result

Thus, the Student Performance Expert System was successfully implemented

Ex No:9	Animal Identification Expert System Using Prolog

Aim

To develop an expert system using Prolog to identify animals based on their characteristics.

Objectives

• To understand rule-based classification.
• To implement knowledge representation in Prolog.
• To identify animals using logical rules.
• To demonstrate inference mechanisms.

Theory

Animal identification systems use observable characteristics such as habitat, body type, and behavior to classify animals.

Problem Description

The problem is to build an expert system that identifies an animal based on its characteristics such as habitat, physical features, and abilities. The system should interact with the user, collect inputs, apply logical rules, and infer the correct animal.

Algorithm

1. Start the program.
2. Ask about animal characteristics.
3. Store responses.
4. Match rules.
5. Display result.
6. Clear responses.

Source Code:

```
:- dynamic known/2.

start :-
write(' ---------------------------------- '), nl,
write(' ANIMAL IDENTIFICATION SYSTEM '), nl,
write(' ---------------------------------- '), nl,
```

```
nl,
identify(Animal),
write('The animal is: '), write(Animal), nl,
undo.

identify(tiger) :-
feature(carnivore),
feature(has_stripes).

identify(elephant) :-
feature(herbivore),
feature(has_trunk).

identify(eagle) :-
feature(can_fly),
feature(has_sharp_vision).

identify(fish) :-
feature(lives_in_water),
feature(has_fins).

identify(frog) :-
feature(lives_in_water),
feature(can_jump).

feature(F) :-
(known(yes, F) -> true ;
known(no, F) -> fail ;
ask(F)).

ask(F) :-
write('Does the animal have '), write(F), write('? (yes/no): '),
read(R), nl,
asserta(known(R, F)),
R = yes.

undo :- retract(known(_, _)), fail.
Undo.
```

Output

```
-----------------------------------
 ANIMAL IDENTIFICATION SYSTEM
-----------------------------------

Does the animal have carnivore? (yes/no): yes.

Does the animal have has_stripes? (yes/no): |: yes.

The animal is: tiger
true .
```

Result

The animal identification expert system successfully identified animals based on user-provided characteristics using logical rules.

Ex No:10	Student Score Prediction Using Linear Regression

Aim

To implement Linear Regression using Python to predict test scores based on hours studied.

Objectives

• To understand the concept of Linear Regression.
• To use scikit-learn for building a regression model.
• To analyze the relationship between study time and test scores.
• To visualize the regression line using Matplotlib.

Theory

Linear Regression is a supervised learning technique used to model the relationship between an independent variable and a dependent variable. It follows the equation y = mx + c, where y is the dependent variable, x is the independent variable, m is the slope, and c is the intercept. The model finds the best-fitting straight line by minimizing prediction errors.

Problem Description

The problem is to predict a student's test score based on the number of hours studied. Using a given dataset, a linear regression model is trained to understand the relationship and predict scores accordingly.

Algorithm

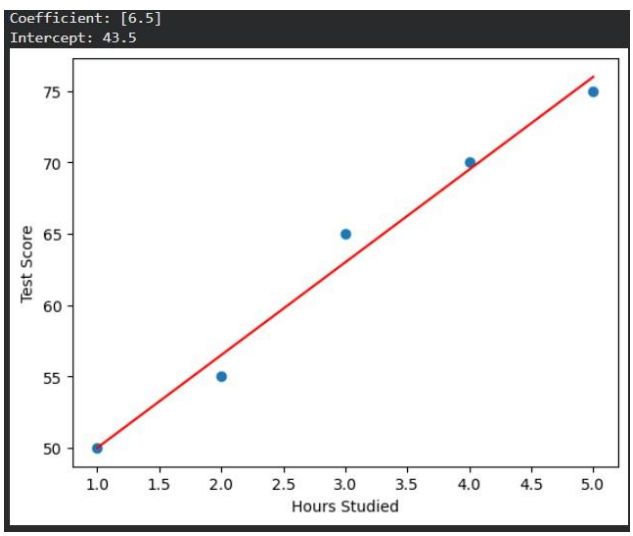
1. Import required libraries.
2. Define input data (hours studied and test scores).
3. Reshape the input data.
4. Create a Linear Regression model.
5. Train the model.
6. Predict scores.
7. Display coefficient and intercept.
8. Plot the graph.

Source Code:

```
import numpy as np
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
```

```
hours_studied = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
test_scores = np.array([50, 55, 65, 70, 75])

model = LinearRegression()
model.fit(hours_studied, test_scores)

predicted_scores = model.predict(hours_studied)

print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)

plt.scatter(hours_studied, test_scores)
plt.plot(hours_studied, predicted_scores, color='red')

plt.xlabel('Hours Studied')
plt.ylabel('Test Score')
plt.show()
```

Output



Result

Thus, the Linear Regression model was successfully implemented and used to predict test scores based on hours studied.

Ex No:11	Iris Flower Classification Using Logistic Regression

Aim

To implement Logistic Regression using Python to classify iris flower species based on input features.

Objectives

• To understand the concept of Logistic Regression.
• To build a classification model using Python.
• To classify data into multiple categories.
• To evaluate model performance using accuracy and classification report.

Theory

Logistic Regression is a supervised machine learning algorithm used for classification problems. It predicts the probability of a categorical dependent variable using a logistic (sigmoid) function. The output is mapped to discrete classes. In multi-class classification, Logistic Regression uses techniques such as one-vs-rest to classify data into more than two categories.

Problem Description

The problem is to develop a classification model using Logistic Regression to identify the species of iris flowers based on their input features such as sepal length, sepal width, petal length, and petal width. The dataset contains multiple classes (setosa, versicolor, and virginica), and the model must learn the relationship between the features and the target labels. The objective is to train the model on a given dataset, test it on unseen data, and accurately classify the iris species while evaluating its performance using appropriate metrics.

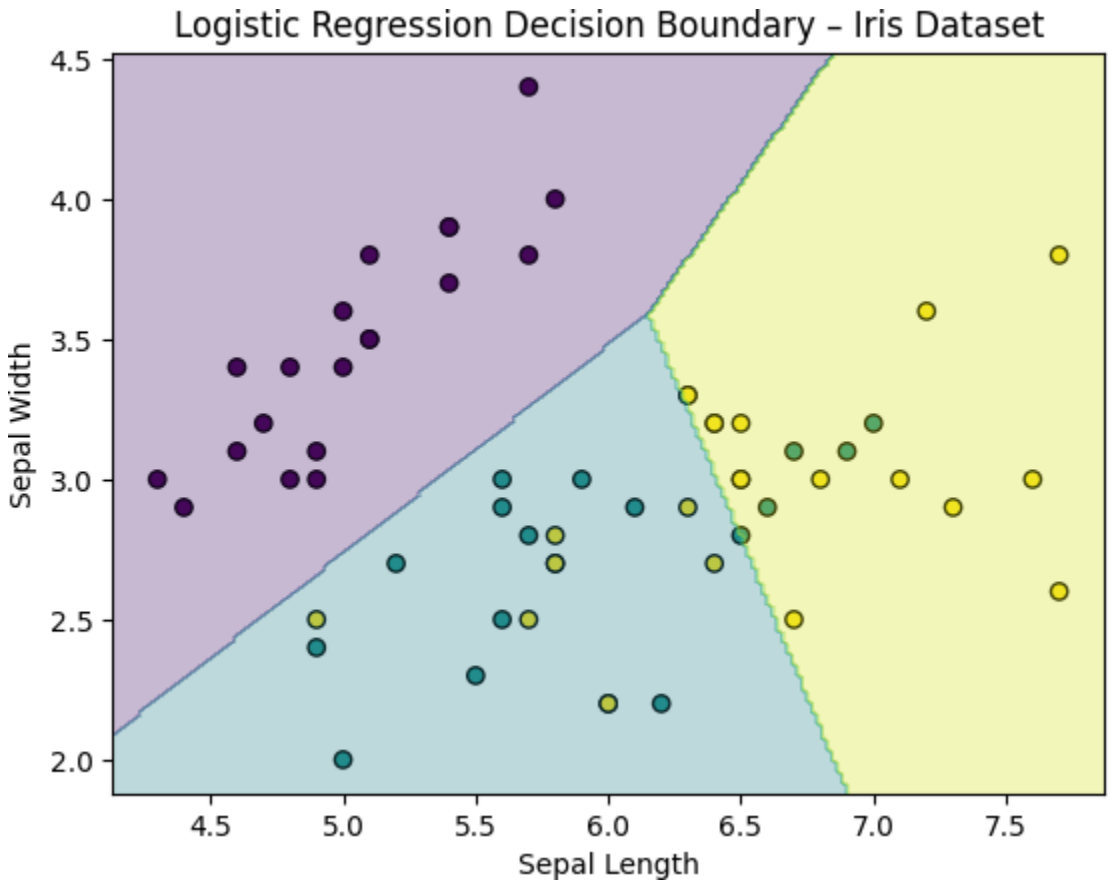
Algorithm

1. Import required libraries.
2. Load the dataset from a CSV file.
3. Convert categorical labels into numerical values.
4. Separate features and target variables.
5. Split the dataset into training and testing sets.
6. Create a Logistic Regression model.
7. Train the model using training data.
8. Predict results on test data.
9. Evaluate the model using accuracy and classification report.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
df = pd.read_csv("iris.csv")

# Encode labels
df['species'] = df['species'].map({
    'setosa': 0,
    'versicolor': 1,
    'virginica': 2
})

# Features and target
X = df.iloc[:, 0:2].values
y = df.iloc[:, -1].values

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output




```
Accuracy: 0.8333333333333334
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         4
           1       0.50      1.00      0.67         2
           2       1.00      0.67      0.80         6

    accuracy                           0.83        12
   macro avg       0.83      0.89      0.82        12
weighted avg       0.92      0.83      0.84        12
```

Result

Thus, the Logistic Regression model was successfully implemented to classify iris flower species, and the performance was evaluated using accuracy and classification metrics

Ex No:12	Customer Purchase Prediction Using Decision Tree

Aim

To implement a Decision Tree classifier using Python to predict whether a customer will buy a computer.

Objectives

• To understand the concept of Decision Trees.
• To perform classification using Decision Tree algorithm.
• To convert categorical data into numerical form.
• To evaluate model performance using accuracy.
• To visualize the decision tree.

Theory

Decision Tree is a supervised machine learning algorithm used for classification and regression. It works by splitting the dataset into subsets based on the most significant feature, forming a tree-like structure. Each internal node represents a decision, each branch represents an outcome, and each leaf node represents a class label.

Problem Description

The problem is to build a model that predicts whether a customer will buy a computer based on features such as age, income, student status, and credit rating. The dataset contains categorical values which must be converted into numerical form before training the model.

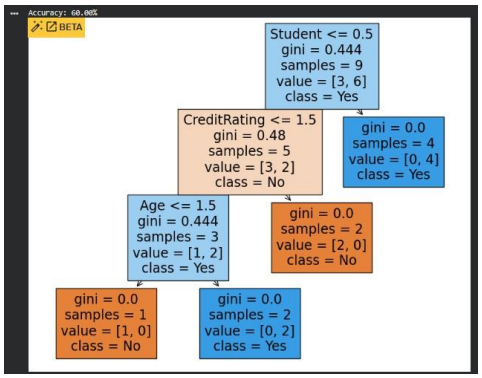
Algorithm

1. Import required libraries.
2. Load the dataset from CSV file.
3. Convert categorical values into numerical values.
4. Define features and target variable.
5. Split dataset into training and testing sets.
6. Create Decision Tree classifier.
7. Train the model using training data.
8. Predict output using test data.
9. Calculate accuracy of the model.
10. Visualize the decision tree.

Source Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn import tree
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('buy_computer.csv')

# Convert categorical data to numerical
df['Age'] = df['Age'].map({'<=30': 1, '31...40': 2, '>40': 3})
df['Income'] = df['Income'].map({'Low': 1, 'Medium': 2, 'High': 3})
df['Student'] = df['Student'].map({'No': 0, 'Yes': 1})
df['CreditRating'] = df['CreditRating'].map({'Fair': 1, 'Excellent': 2})
df['BuysComputer'] = df['BuysComputer'].map({'No': 0, 'Yes': 1})
# Features and target
X = df[['Age', 'Income', 'Student', 'CreditRating']]
y = df['BuysComputer']
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)
# Create model
clf = DecisionTreeClassifier()
# Train model
clf.fit(X_train, y_train)
# Predict
y_pred = clf.predict(X_test)
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Plot tree
plt.figure(figsize=(10,8))
tree.plot_tree(clf, filled=True,
               feature_names=['Age','Income','Student','CreditRating'],
               class_names=['No','Yes'])
plt.show()
```

Output



Result

Thus, the Decision Tree classifier was successfully implemented and visualized for predicting whether a customer will buy a computer.

Ex No:13	Iris Flower Classification Using Support Vector Machine

Aim

To implement a Support Vector Machine (SVM) classifier using Python to classify the Iris dataset.

Objectives

• To understand the concept of Support Vector Machines.
• To perform classification using SVM.
• To split dataset into training and testing sets.
• To evaluate model performance using accuracy and classification report.

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the optimal hyperplane that separates data points of different classes with maximum margin. In this experiment, SVM is applied to the Iris dataset to classify different species of flowers based on their features.

Problem Description

The problem is to build a machine learning model that classifies iris flowers into different species using their features. The system should load the dataset, train an SVM model, test it on unseen data, and evaluate its performance.

Algorithm

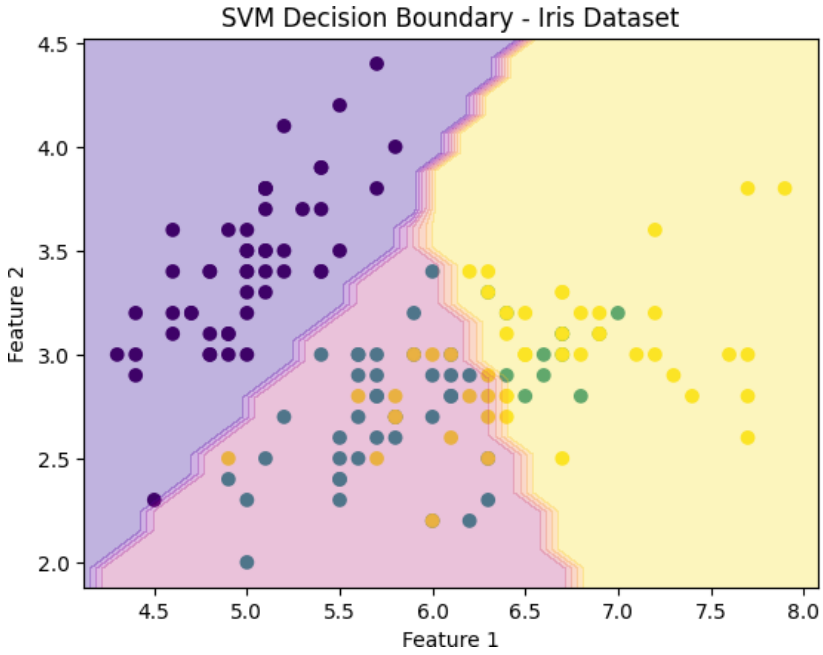
1. Import required libraries.
2. Load the Iris dataset.
3. Select input features and target labels.
4. Split dataset into training and testing sets.
5. Create SVM classifier model.
6. Train the model using training data.
7. Predict output using test data.
8. Evaluate performance using accuracy and classification report.

Source Code:

```
# Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
```

```
from sklearn import svm
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = datasets.load_iris()
X = iris.data[:, :2]  # first two features
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create SVM model
model = svm.SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)
# Predict
y_pred = model.predict(X_test)
# Evaluate
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output



```
•••  Accuracy: 0.9
                  precision    recall  f1-score   support

              0       1.00      1.00      1.00        10
              1       0.88      0.78      0.82         9
              2       0.83      0.91      0.87        11

       accuracy                           0.90        30
      macro avg       0.90      0.90      0.90        30
   weighted avg       0.90      0.90      0.90        30
```

Result

Thus, the Support Vector Machine classifier was successfully implemented and evaluated using the Iris dataset.

Ex No:14	Customer Segmentation Using K-Means Clustering

Aim

To implement K-Means Clustering using Python to group customers based on income and spending score.

Objectives

• To understand the concept of clustering.
• To implement K-Means clustering using Python.
• To group similar data points into clusters.
• To visualize clusters using a graph.

Theory

K-Means Clustering is an unsupervised machine learning algorithm used to group similar data points into clusters. It works by dividing the dataset into k clusters, where each data point belongs to the cluster with the nearest mean (centroid). The algorithm iteratively updates cluster centroids until convergence is achieved. It is widely used for customer segmentation and pattern recognition.

Problem Description

The problem is to group customers based on their annual income and spending score. Using a small dataset, the K-Means algorithm is applied to divide customers into clusters such that customers in the same group have similar characteristics.

Algorithm

1. Import required libraries.
2. Create or load the dataset.
3. Select relevant features.
4. Apply K-Means clustering algorithm.
5. Assign cluster labels to data points.
6. Print clustered data.
7. Plot clusters and centroids.

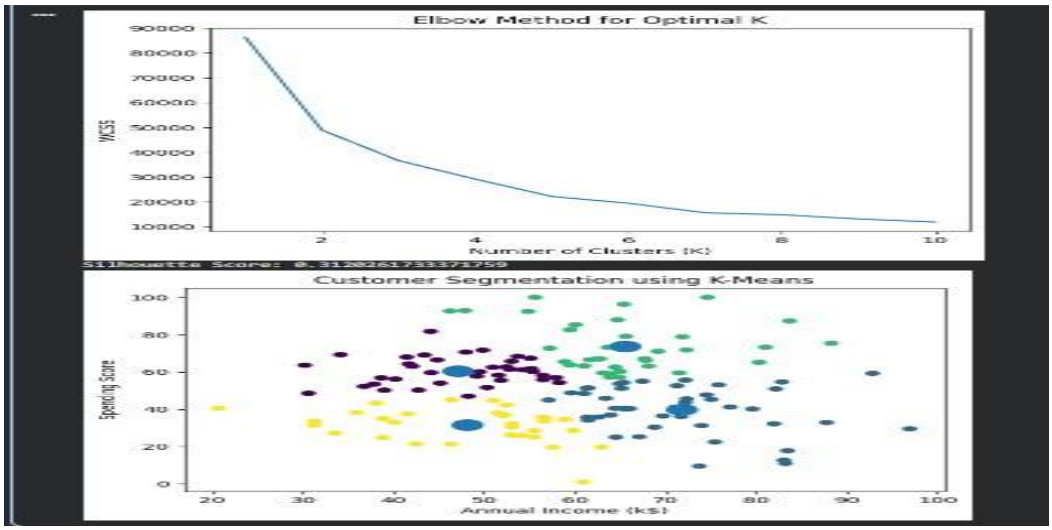
Source Code:

```
# Import libraries
import pandas as pd                    # For reading dataset
import matplotlib.pyplot as plt        # For plotting graphs
from sklearn.cluster import KMeans     # K-Means clustering algorithm
```

```
from sklearn.metrics import silhouette_score

df = pd.read_csv("customer_kmeans_dataset.csv")  # Load the dataset

# Select features for clustering
# We use Income and Spending Score
X = df[['Annual_Income_k$', 'Spending_Score_1_100']].values

wcss = []  # WCSS = Within Cluster Sum of Squares (measure of cluster error)

# Test different numbers of clusters
for i in range(1, 11):                    # Try cluster numbers from 1 to 10

    kmeans = KMeans(n_clusters=i, random_state=42)  # Create KMeans model

    kmeans.fit(X)                         # Train the model using dataset

    wcss.append(kmeans.inertia_)          # Store WCSS value (cluster error)

# Plot Elbow Graph
plt.plot(range(1, 11), wcss)              # X-axis = number of clusters, Y-axis = WCSS
plt.title("Elbow Method for Optimal K")     # Graph title
plt.xlabel("Number of Clusters (K)")        # X-axis label
plt.ylabel("WCSS")                          # Y-axis label
plt.show()                                  # Display graph

# Assume elbow graph suggests K = 5
kmeans = KMeans(n_clusters=4, random_state=42)

y_kmeans = kmeans.fit_predict(X)            # Train model and assign clusters


score = silhouette_score(X, y_kmeans)       # Calculate silhouette score

print("Silhouette Score:", score)           # Print clustering quality

# Plot clustered data
plt.scatter(X[:,0], X[:,1], c=y_kmeans)     # Color points by cluster

# Plot centroids (cluster centers)
plt.scatter(kmeans.cluster_centers_[:,0],
            kmeans.cluster_centers_[:,1],
```

```
        s=200)                    # Larger size for visibility

plt.title("Customer Segmentation using K-Means")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score")
plt.show()
```

Output



Result

Thus, the K-Means clustering algorithm was successfully implemented to group customers into clusters based on their income and spending score.